R. K. COLLEGE
BCA 2ND YER
CC5: PYTHON PROGRAMMING

UNIT-3 LISTS , TUPLES AND DICTIONARIES

OPERATOR WORK ON LIST IN PYTHON

In Python, the in operator determines whether a given value is a constituent element of a sequence such as a string, array, list, or tuple.

When used in a condition, the statement returns a Boolean result of True or False. The statement returns True if the specified value is found within the sequence. When it is not found, we get a False.

we will show how the in operator works on a list in different ways using Python. Here we will see 3 different scenarios −

- To find a single element/object in a single list/flat list.
- To find multiple elements list in a nested list.
- Usage of in operator with if statement.

```
# input list
lst = ["Hello", 10, "TutorialsPoint", 20, "python", "code"]


# Checking if {TutorialsPoint} element in list using in operator
print("TutorialsPoint" in lst)


# Checking if {bigdata} element in list using in operator
print("bigdata" in lst)
```

Output

On executing, the above program will generate the following output −

True

False

Method 2: To find multiple elements list in a nested

Algorithm (Steps)

Following are the Algorithm/steps to be followed to perform the desired task −

- Create a variable to store the input nested list and give it some random list values.
- Check if the list is present in the nested list or not using in operator and print it.

The following program checks whether the given list is present in the nested list or not using the **in** operator −

```
# input list
lst = [["Hello", 10], ["TutorialsPoint", 20], ["python", "code"]]


# Checking if {TutorialsPoint,20} elements list present in nested list using in operator
print(["TutorialsPoint",20] in lst)


# Checking if {TutorialsPoint,code} elements list present in nested list using in operator
print(["TutorialsPoint","code"] in lst)
```

Output

On executing, the above program will generate the following output −

True

False

Method 3: Usage of in operator with if statement.

Algorithm (Steps)

Following are the Algorithm/steps to be followed to perform the desired task −

- Create a variable to store the input nested list and give it some random list values.
- Check if the element is present in the list or not using the in operator and if statement.
- The if statement executes only if the result returned by the in operator is true i.e if the element is present in the list.
- Print the result tuple after conversion from a list into a tuple.
- Print the data type of the result tuple using the type() function for verification.Write the else statement if the element is not present in the list.

The following program checks whether the single element is present in the flat list or not using the **in** operator −

```
# input list
lst = ["Hello", 10, "TutorialsPoint", 20, "python", "code"]


# Checking if {TutorialsPoint} element in list using in operator
if "TutorialsPoint" in lst:
   print('{TutorialsPoint} Element is in the given list')
```

```
# Checking if {bigdata} element in list using in operator
if "bigdata" in lst:
   print('{bigdata} Element is in the given list')

# If {bigdata} is not in list
else:
   print('{bigdata} Element is not present in the given list')
```

Output

On executing, the above program will generate the following output −

{TutorialsPoint} Element is in the given list

{bigdata} Element is not present in the given list

BUILT-IN LIST FUNCTIONS & METHODS IN PYTHON

- **Python includes the following list functions −**

Sr.No	Function with Description
1	**range()**Gives the **range** of the list.
2	len(list)Gives the total length of the list.p>
3	max(list)Returns item from the list with max value.
4	min(list)Returns item from the list with min value.
5	list(seq)Converts a tuple into list.

- **Python includes following list methods**

Sr.No	Methods with Description
1	list.append(obj)Appends object obj to list
2	list.count(obj)Returns count of how many times obj occurs in list

Sr.No	Methods with Description
3	list.extend(seq)Appends the contents of seq to list
4	list.index(obj)Returns the lowest index in list that obj appears
5	list.insert(index, obj)Inserts object obj into list at offset index
6	list.pop(obj=list[-1])Removes and returns last object or obj from list
7	list.remove(obj)Removes object obj from list
8	list.reverse()Reverses objects of list in place
9	list.sort([func])Sorts objects of list, use compare func if given

1. **range():- Gives the range of the list.**

The Python list function **range()** takes in some integers as arguments and generates a list of integers.
Syntax:
range([start,]stop[,step])

Where:
- **start**: Specifies where to start generating integers for the list.
- **stop**: Specifies where to stop generating integers for the list.
- **step**: Specifies the incrementation.

From the syntax above, start and step are both optional and they default to 0 and 1 respectively.

Example 1: Create a sequence of numbers from 4 to 20, but increment by 2 and print it.

```
def create_seq(start, end, step):
    # create a range object
    r = range(start, end, step)
    # print items in the range object.
    for item in r:
        print(item)


if __name__ == '__main__':
    start = 4  # define our start number
    end = 20   # define out end number
    step = 2   # define out step number

    print("Range of numbers:")
```

```
create_seq(start, end, step)
```

Output:

```
>>> list(range(4,20,2))
[4, 6, 8, 10, 12, 14, 16, 18]
```

2. **len(list)Gives the total length of the list.p>**

The Python list method l**en()** returns the size(number of items) of the list by calling the list object's own length method. It takes in a list object as an argument and doesn't have a side effect on the list.
Syntax:

```
len(s)
```

Example 1: Write a function that computes and returns the size/length of a list.

```
def get_len(l):
    # Python list function len() computes the size of the list.
    return len(l)

if __name__ == '__main__':
    l1 = []                 # defined an empty list
    l2 = [5,43,6,1]         # define a list of 4 elements
    l3 = [[4,3],[0,1],[3]]  # define a list of 3 elements(lists)

    print("L1 len: ", get_len(l1))
    print("L2 len: ", get_len(l2))
    print("L3 len: ", get_len(l3))
```

3. **min(list):-Returns item from the list with min value..**

The Python **min()** function returns the smallest item in a sequence.
Syntax:
min(iterable[,key, default])

Where:

- **iterable** here will be a list of items.

- **key** here specifies a function of one argument that is used to extract a comparison key from each list element.
- **default** here specifies a value that will be returned if the iterable is empty.

Example 6: Find the smallest number in the list [4,3,9,10,33,90].

```
>>> numbers = [4,3,9,10,33,90]
>>> min(numbers)
3
```

4. **max(list):- Returns item from the list with max value.**

The Python **max()** function returns the highest item in a sequence.
Syntax:
max(iterable[,key, default])

Where:

- **iterable** here will be a list of items.
- **key** here specifies a function of one argument that is used to extract a comparison key from each list element.
- **default** here specifies a value that will be returned if the iterable is empty.
-

Example 8: Find the largest number in the list [4,3,9,10,33,90].

```
>>> numbers = [4,3,9,10,33,90]
>>> max(numbers)
90
```

5. **list(seq):- Converts a tuple into list.**

Example 4: Sum items from a list

```
>>>  sum([9,3,2,5,1,-9])
11
```

Example 5: Start with 9 and add all items from the list [9,3,2,5,1,-9].

```
>>> sum([9,3,2,5,1,-9], 9)
20
```

Example 6: Sort the list [4,3,10,6,21,9,23] in descending order.

```
>>> numbers = [4,3,10,6,21,9,23]
>>> sorted(numbers, reverse=True)
[23, 21, 10, 9, 6, 4, 3]
```

- **METHODS**

1. list.append(obj)Appends object obj to list

fruits = ["apple", "banana", "cherry

fruits.append("orange")

print(fruits)

output:

2. list.count(obj)Returns count of how many times obj occurs in list

fruits = ["apple", "banana", "cherry"]

x = fruits.count("cherry")

print(x)

output: 1

3. list.extend(seq)Appends the contents of seq to list

fruits = ['apple', 'banana', 'cherry']

cars = ['Ford', 'BMW', 'Volvo']

fruits.extend(cars)

print(fruits)

output: ['apple', 'banana', 'cherry', 'Ford', 'BMW', 'Volvo']

4. list.index(obj)Returns the lowest index in list that obj appears

```
fruits = ['apple', 'banana', 'cherry']
x = fruits.index("cherry")
print(x)
output:- 2
```

5. list.insert(index, obj)Inserts object obj into list at offset index

```
fruits = ['apple', 'banana', 'cherry']
fruits.insert(1, "orange")
print(fruits)
output: ['apple', 'orange', 'banana', 'cherry']
```

6. list.pop(obj=list[-1])Removes and returns last object or obj from list

```
fruits = ['apple', 'banana', 'cherry']
fruits.pop(1)
print(fruits)
output: ['apple', 'cherry']
```

7. list.remove(obj)Removes object obj from list

```
fruits = ['apple', 'banana', 'cherry']
fruits.remove("banana")
print(fruits)
output:-
```

8. list.reverse()Reverses objects of list in place

```
fruits = ['apple', 'banana', 'cherry']
```

```
fruits.reverse()
print(fruits)
```

output:-

```
['cherry', 'banana', 'apple']
```

9. list.sort([func])Sorts objects of list, use compare func if given

```
cars = ['Ford', 'BMW', 'Volvo']
cars.sort()
print(cars)
```

output:-

```
['BMW', 'Ford', 'Volvo']
```